Numerical Analysis: Final exam (**70 marks**)

Solutions

Urbain Vaes

Fall $(11111101001)_2$

Question 1 (Floating point arithmetic, **10 marks**)**.** True or false?

1. Let $(\bullet)_2$ denote binary representation. Then $(1000000000)_2 \times (0.001)_2 = (1000000)_2$.
2. It holds that $(0.111111)_2 + (0.000001)_2 = (1.001)_2$.
3. It holds that $(0.\overline{1100})_2 = \frac{4}{5}$.
4. Let ε_{32} denote the machine epsilon for the `Float32` format, i.e. `eps(Float32)`. Then the number 3 is representable exactly in this format, and the next representable number in the `Float32` format is $3 + 2\varepsilon_{32}$.
5. In Julia, `Float64(3.25) == Float32(3.25)` evaluates to `true`.
6. The spacing (in absolute value) between successive half-precision (`Float16`) floating point numbers is always greater than or equal to `eps(Float16)`.
7. In Julia, define `f(x) = x == Inf ? - x : f(-2x)`. Then `f(5.0)` evaluates to `Inf`.
8. There exists $n \in \mathbf{N}$ such that the machine epsilon for the `Float64` format equals 2^{-n}. In other words, the machine epsilon is an exact negative power of two.
9. In Julia, it holds that `eps()/3 + eps()/3 + 1 > 1` evaluates to `true`.
10. In Julia, the expression `1 + 1024eps() == 2` evaluates to `true`.
11. **Bonus.** Does the Julia expression `log2(2 + eps()) == 2` evaluate to `true` or `false`.
12. **Bonus.** Does the Julia expression `sqrt(4 + 5eps()) == 2` evaluate to `true` or `false`?

Solution. Below, $\varepsilon = \varepsilon_{64} = 2^{-52}$ denotes the machine epsilon for `Float64`.

1. **True**: $(1000000000)_2 = 2^9$ and $(0.001)_2 = 2^{-3}$, so the product is $2^6 = (1000000)_2$.
2. **False**: $(0.111111)_2 + (0.000001)_2 = (1.000000)_2 = 1$, whereas $(1.001)_2 = 1 + \frac{1}{8}$.
3. **True**: if $x = (0.\overline{1100})_2$, then $2^4 x = (1100)_2 + x = 12 + x$, so $x = \frac{12}{15} = \frac{4}{5}$.
4. **True**: $3 = 2^1 \times (1.1)_2$ is representable, and the spacing between successive `Float32` numbers in $[2, 4)$ is $2\varepsilon_{32}$.

5. **True**: $3.25 = (11.01)_2$ is representable exactly in both formats, so both sides represent the same number.
6. **False**: the spacing between successive `Float16` numbers in $[2^e, 2^{e+1})$ is $2^e \times$`eps(Float16)`, which is smaller than `eps(Float16)` for $e < 0$. For example, the spacing between 0 and `nextfloat(Float16(0))` $\approx 6 \times 10^{-8}$ is much smaller than `eps(Float16)` $= 2^{-10} \approx 10^{-3}$.
7. **False**: the successive arguments are $5, -10, 20, -40, \ldots$, until $-2x$ overflows to `Inf` or `-Inf`. If the argument is `Inf`, the function returns `-Inf`; if it is `-Inf`, the function calls `f(Inf)`, which also returns `-Inf`. Therefore `f(5.0)` evaluates to `-Inf`.
8. **True**: $\varepsilon_{64} = 2^{-52}$.
9. **True**: the expression is evaluated from left to right, and `eps()/3 + eps()/3` $\approx \frac{2}{3}\varepsilon$. Since $1 + \frac{2}{3}\varepsilon$ is closer to $1 + \varepsilon$ than to 1, the result of the addition is rounded to $1 + \varepsilon > 1$.
10. **False**: $1 + 1024\varepsilon = 1 + 2^{-42}$ is exactly representable and different from 2.
11. **False**: even with exact arithmetic, $\log_2(2+\varepsilon) \approx \log_2(2) = 1 \neq 2$. In floating point arithmetic, $2 + \varepsilon$ lies exactly halfway between 2 and $2 + 2\varepsilon$, and is rounded to 2 (round to even), so `log2(2 + eps())` returns exactly 1.
12. **True**: the spacing between successive `Float64` numbers in $[4, 8)$ is 4ε, so $4 + 5\varepsilon$ is rounded to $4 + 4\varepsilon$. The exact square root of this number is $2\sqrt{1+\varepsilon} = 2 + \varepsilon - \frac{\varepsilon^2}{4} + \cdots$, which is slightly smaller than the midpoint $2 + \varepsilon$ between 2 and the next `Float64` number $2 + 2\varepsilon$. Since `sqrt` is correctly rounded, the result is exactly 2.

△

Question 2 (Interpolation, **10 marks**)**.** The first part of this question is an open-ended question, while the second part is a true/false.

1. (**5 marks**, open-ended question) Describe the Gregory–Newton method to calculate the interpolation polynomial in the case of equally spaced interpolation nodes at $\{0, 1, 2, \cdots, n\}$ and corresponding values $\{f(0), f(1), f(2), \cdots, f(n)\}$. In particular, address the following points:
   - Write down the expression for the interpolating polynomial;
   - Discuss the advantages and disadvantages of this interpolation method compared to Lagrange interpolation;
   - Illustrate the method on a concrete example of your choice.
2. (**5 marks**) Are the following assertions true or false?
   - Let $f(x) = 2x^4 + x^2 - 3x - 1$, and let $f_3 \in \mathcal{P}_3$ denote the polynomial interpolant of f at four points $x_0 < x_1 < x_2 < x_3$. Then
   $$f(x) - f_3(x) = \frac{2}{4!}(x - x_0)(x - x_1)(x - x_2)(x - x_3)\,.$$
   - Let $f\colon \mathbf{R} \to \mathbf{R}$ be the Runge function defined by $f(x) = (1 + 25x^2)^{-1}$, and let $f_n \in \mathcal{P}_n$ denote the polynomial interpolating f at $n+1$ equally spaced points in the interval $[-1, 1]$.

Then, it holds that

$$\lim_{n\to\infty}\left(\max_{x\in[-1,1]}\left|f(x)-f_n(x)\right|\right)=0\,.$$

- Let $f(x)=\sin(\pi x)+\mathrm{e}^x$, and let f_n denote the piecewise linear interpolation of f at $n+1$ equally spaced points in the interval $[-1,1]$. Then, it holds that

$$\lim_{n\to\infty}\left|\int_{-1}^{1} f(x)-f_n(x)\,\mathrm{d}x\right|=0\,.$$

- Consider the function $\boldsymbol{F}\colon \mathbf{R}^{n+1}\to\mathbf{R}$ defined by

$$F(\boldsymbol{x}) \coloneqq \max_{z\in[-1,1]}\left|(z-x_0)(z-x_1)\cdots(z-x_n)\right|, \qquad \boldsymbol{x}=(x_0,x_1,\ldots,x_n)\in\mathbf{R}^{n+1}\,.$$

  Then $\boldsymbol{F}$ is *maximized (as large as possible)* when the points $x_0,x_1,\ldots,x_n$ are chosen as the roots of the Chebychev polynomial of degree $n+1$.

- In the following code, `p` is the interpolating polynomial through the data in `x` and `y`. Here, for an array `y` of size n, the command `diff(y)` returns the array of size $n-1$ whose i-th entry is equal to `y[i+1] - y[i]`.

```
x = [0, 1, 2, 3]
y = [1, 2, 1, 2]

function p(x)
    return (y[1]
            + diff(y)[1] * x
            + 1/2 * diff(diff(y))[1] * x^2
            + 1/6 * diff(diff(diff(y)))[1] * x^3)
end
```

- (**Bonus, 1 mark**) Let $f(n)=2^n$. Then for all $n\in\mathbf{N}$, it holds that

$$f(n)=\sum_{i=0}^{+\infty}\frac{\Delta^i f(0)}{i!}n^{\underline{i}},$$

  where Δ denotes the forward difference operator, meaning that $\Delta f(n)=f(n+1)-f(n)$, Justify your answer.

Solution. 1. Many answers are possible; a good answer should contain the following elements. The forward differences of f are defined by $\Delta f(x)=f(x+1)-f(x)$ and $\Delta^{i+1}f=\Delta(\Delta^i f)$, and the falling factorials by $x^{\underline{0}}=1$ and $x^{\underline{i}}=x(x-1)\cdots(x-i+1)$. The Gregory–Newton formula expresses the interpolating polynomial as

$$p(x)=\sum_{i=0}^{n}\frac{\Delta^i f(0)}{i!}x^{\underline{i}}.$$

This is the Newton form of the interpolating polynomial, where the divided differences reduce to $\frac{\Delta^i f(0)}{i!}$ because the nodes are equally spaced. The coefficients are obtained from a table of differences, in $\mathcal{O}(n^2)$ operations.

- *Advantages*: adding a node $n+1$ only requires computing one new diagonal of the table of differences and adding one term to the sum, whereas all the Lagrange basis polynomials change. The polynomial can be evaluated in $\mathcal{O}(n)$ operations using a nested (Horner-like) scheme.
- *Disadvantages*: the method applies only to equally spaced nodes. Since the interpolating polynomial is unique, the approximation is the same as with Lagrange interpolation and suffers from the same issues (Runge phenomenon for equally spaced nodes). High-order differences are sensitive to round-off errors and to noise in the data.

Example: for the data $f(0), f(1), f(2), f(3) = 1, 2, 1, 2$, the table of differences is

x	f	Δf	$\Delta^2 f$	$\Delta^3 f$
0	1	1	-2	4
1	2	-1	2	
2	1	1		
3	2			

and so $p(x) = 1 + x - x(x-1) + \frac{2}{3}x(x-1)(x-2)$. We check that $p(0) = 1$, $p(1) = 2$, $p(2) = 1$ and $p(3) = 1 + 3 - 6 + 4 = 2$.

2.
- **False**: the interpolation error is $f(x) - f_3(x) = \frac{f^{(4)}(\xi)}{4!}\prod_{i=0}^{3}(x - x_i)$, and $f^{(4)} = 48$, so $f(x) - f_3(x) = 2\prod_{i=0}^{3}(x - x_i)$.
- **False**: this is the Runge phenomenon: for equally spaced nodes, the interpolation error for the Runge function diverges as $n \to \infty$.
- **True**: for a continuous function on a closed interval, the piecewise linear interpolant converges uniformly (here f is smooth and the error is bounded by $\frac{h^2}{8}\max|f''|$, with $h = \frac{2}{n}$), so the integral of the error converges to 0.
- **False**: the roots of the Chebyshev polynomial of degree $n+1$ *minimize* F.
- **False**: the code uses the monomials x^i instead of the falling factorials $x^{\underline{i}}$. Here `diff(y)[1]` $= 1$, `diff(diff(y))[1]` $= -2$ and `diff(diff(diff(y)))[1]` $= 4$, so $p(x) = 1 + x - x^2 + \frac{2}{3}x^3$, and $p(1) = \frac{5}{3} \neq 2$.
- **True**: since $\Delta f(n) = 2^{n+1} - 2^n = f(n)$, it holds that $\Delta^i f(0) = f(0) = 1$ for all i. For $n \in \mathbf{N}$, we have $n^{\underline{i}} = 0$ for $i > n$, so the right-hand side is

$$\sum_{i=0}^{n} \frac{n^{\underline{i}}}{i!} = \sum_{i=0}^{n} \binom{n}{i} = 2^n.$$

(Equivalently, the sum truncated at $i = n$ is the polynomial interpolating f at $0, \ldots, n$, evaluated at n.)

△

Question 3 (Numerical interpolation, **10 marks**)**.** We wish to derive and implement Boole's rule for numerical integration. Boole's rule is a Newton–Cotes quadrature rule based on five equally

spaced points in the interval $[-1, 1]$.

$$I[u] \coloneqq \int_{-1}^{1} u(x)\,\mathrm{d}x \approx w_0 u(-1) + w_1 u\left(-\frac{1}{2}\right) + w_2 u(0) + w_3 u\left(\frac{1}{2}\right) + w_4 u(1) =: \widehat{I}_5[u]. \tag{1}$$

1. (**5 marks**) Calculate the weights w_0, w_1, w_2, w_3, w_4 to ensure that the quadrature rule $\widehat{I}_5$ is exact for all polynomials in $\mathcal{P}_d$, where d is as large as possible. Include the details of your calculation.

   ***Hint**: It is useful to require that the quadrature rule is exact for the Lagrange basis polynomials.*

   ***Hint**: By symmetry, you can expect that $w_0 = w_4$ and $w_1 = w_3$.*

2. (**1 mark**) What is the degree of precision of Boole's rule (1)?

3. (**4 marks**) Write a function `boole(u, a, b)` that returns, using Boole's rule appropriately adapted to the interval $[a, b]$, an approximation of the integral

$$\int_a^b u(x)\,\mathrm{d}x\,. \tag{2}$$

4. (**Bonus**, **2 marks**) Write a function `composite_boole(u, a, b, N)` that returns an approximation of the integral (2), this time using a composite version of Boole's rule. More precisely, the approximation should be obtained by partitioning the integration interval $[a, b]$ into N cells, and applying Boole's rule within each cell. Your function should call the function `boole` you wrote in the previous item.

Solution. 1. By symmetry, odd monomials are integrated exactly provided that $w_0 = w_4$ and $w_1 = w_3$. Requiring that the rule be exact for 1, x^2 and x^4 gives

$$\begin{aligned} 2w_0 + 2w_1 + w_2 &= 2, \\ 2w_0 + \frac{2}{4}w_1 &= \frac{2}{3}, \\ 2w_0 + \frac{2}{16}w_1 &= \frac{2}{5}. \end{aligned}$$

Subtracting the third equation from the second gives $\frac{3}{8}w_1 = \frac{4}{15}$, i.e. $w_1 = \frac{32}{45}$. Then $w_0 = \frac{1}{3} - \frac{1}{4}w_1 = \frac{7}{45}$, and $w_2 = 2 - 2w_0 - 2w_1 = \frac{12}{45}$. Therefore

$$(w_0, w_1, w_2, w_3, w_4) = \frac{1}{45}(7, 32, 12, 32, 7).$$

(Equivalently, $w_i = \int_{-1}^{1} \ell_i(x)\,\mathrm{d}x$, where ℓ_i are the Lagrange basis polynomials associated with the nodes.)

2. The rule is exact for all polynomials of degree 5: for degree at most 4 by construction, and for x^5 by symmetry. For x^6, it gives $2 \times \frac{7}{45} + 2 \times \frac{32}{45} \times \frac{1}{64} = \frac{1}{3} \neq \frac{2}{7}$. The degree of precision is therefore 5.

3. With $h = \frac{b-a}{4}$, the change of variable $x = \frac{a+b}{2} + \frac{b-a}{2}t$ gives

```
function boole(u, a, b)
```

```
        h = (b - a) / 4
        return (b - a) / 90 * (7u(a) + 32u(a + h) + 12u(a + 2h) + 32u(a + 3h) + 7u(b))
    end
```

4.
```
    function composite_boole(u, a, b, N)
        x = range(a, b, length = N + 1)
        return sum(boole(u, x[i], x[i+1]) for i in 1:N)
    end
```

△

Question 4 (Iterative method for linear systems, **10 marks**)**.** Assume that $\mathsf{A} \in \mathbf{R}^{n\times n}$ is a nonsingular matrix and that $\boldsymbol{b} \in \mathbf{R}^n$. We wish to solve the linear system

$$\mathsf{A}\boldsymbol{x} = \boldsymbol{b} \tag{3}$$

using an iterative method where each iteration is of the form

$$\mathsf{M}\boldsymbol{x}_{k+1} = \mathsf{N}\boldsymbol{x}_k + \boldsymbol{b}. \tag{4}$$

Here $\mathsf{A} = \mathsf{M} - \mathsf{N}$ is a splitting of A such that M is nonsingular, and $\boldsymbol{x}_k \in \mathbf{R}^n$ denotes the k-th iterate of the numerical scheme.

1. (**3 marks**) Let $\boldsymbol{e}_k \coloneqq \boldsymbol{x}_k - \boldsymbol{x}_*$, where $\boldsymbol{x}_*$ is the exact solution to (3). Prove that the error satisfies the recurrence relation
$$\boldsymbol{e}_{k+1} = \mathsf{M}^{-1}\mathsf{N}\boldsymbol{e}_k.$$

2. (**3 marks**) Let $L = \|\mathsf{M}^{-1}\mathsf{N}\|_1$. Prove that
$$\forall k \in \mathbf{N}, \qquad \|\boldsymbol{e}_k\|_1 \leqslant L^k \|\boldsymbol{e}_0\|_1. \tag{5}$$

3. (**1 mark**) In general, is the condition $\|\mathsf{M}^{-1}\mathsf{N}\|_1 < 1$ sufficient for convergence (that it, does it ensure that $\boldsymbol{x}_k \to \boldsymbol{x}_*$ for every $\boldsymbol{x}_0$)? Justify briefly your answer.

4. (**1 mark**) In general, is the condition $\|\mathsf{M}^{-1}\mathsf{N}\|_1 < 1$ necessary for convergence? Justify briefly your answer.

5. (**2 marks**) Let $\boldsymbol{r}_k \coloneqq \mathsf{A}\boldsymbol{x}_k - \boldsymbol{b}$ denote the residual at iteration k. Prove that
$$\boldsymbol{r}_{k+1} = \mathsf{N}\mathsf{M}^{-1}\boldsymbol{r}_k\,.$$

Solution.

1. Since $\mathsf{A}\boldsymbol{x}_* = \boldsymbol{b}$, the exact solution satisfies $\mathsf{M}\boldsymbol{x}_* = \mathsf{N}\boldsymbol{x}_* + \boldsymbol{b}$. Subtracting this equation from (4), we obtain $\mathsf{M}\boldsymbol{e}_{k+1} = \mathsf{N}\boldsymbol{e}_k$, and so $\boldsymbol{e}_{k+1} = \mathsf{M}^{-1}\mathsf{N}\boldsymbol{e}_k$ since M is nonsingular.

2. By definition of the subordinate matrix norm $\|\bullet\|_1$,
$$\|\boldsymbol{e}_{k+1}\|_1 = \|\mathsf{M}^{-1}\mathsf{N}\boldsymbol{e}_k\|_1 \leqslant \|\mathsf{M}^{-1}\mathsf{N}\|_1 \|\boldsymbol{e}_k\|_1 = L\|\boldsymbol{e}_k\|_1.$$

The inequality (5) follows by induction, since it holds trivially for $k = 0$.

3. Yes: if $L < 1$, then $L^k \to 0$, so $\|\boldsymbol{e}_k\|_1 \to 0$ by (5) for every $\boldsymbol{x}_0$.
4. No: the method converges for every $\boldsymbol{x}_0$ if and only if the spectral radius $\rho(\mathsf{M}^{-1}\mathsf{N})$ is less than 1, and $\rho(\mathsf{M}^{-1}\mathsf{N}) \leqslant \|\mathsf{M}^{-1}\mathsf{N}\|_1$ with strict inequality in general. For example, if $\mathsf{M}^{-1}\mathsf{N} = \begin{pmatrix} 0 & 2 \\ 0 & 0 \end{pmatrix}$, then $\|\mathsf{M}^{-1}\mathsf{N}\|_1 = 2$, but $(\mathsf{M}^{-1}\mathsf{N})^2 = 0$, so $\boldsymbol{e}_k = 0$ for all $k \geqslant 2$.
5. Using $\mathsf{A} = \mathsf{M} - \mathsf{N}$ and (4),

$$\boldsymbol{r}_{k+1} = \mathsf{M}\boldsymbol{x}_{k+1} - \mathsf{N}\boldsymbol{x}_{k+1} - \boldsymbol{b} = \mathsf{N}\boldsymbol{x}_k - \mathsf{N}\boldsymbol{x}_{k+1} = \mathsf{N}(\boldsymbol{x}_k - \boldsymbol{x}_{k+1}).$$

On the other hand, $\mathsf{M}(\boldsymbol{x}_{k+1} - \boldsymbol{x}_k) = \mathsf{N}\boldsymbol{x}_k + \boldsymbol{b} - \mathsf{M}\boldsymbol{x}_k = -\boldsymbol{r}_k$, so $\boldsymbol{x}_k - \boldsymbol{x}_{k+1} = \mathsf{M}^{-1}\boldsymbol{r}_k$. Therefore $\boldsymbol{r}_{k+1} = \mathsf{N}\mathsf{M}^{-1}\boldsymbol{r}_k$.

△

Implementation exercise 1 (Linear systems, **10 marks**)**.** Assume that $\mathsf{A} \in \mathbf{R}^{n\times n}$ is an *upper triangular* matrix with nonzero diagonal entries (hence invertible) and that $\boldsymbol{b} \in \mathbf{R}^n$. We wish to solve the linear system

$$\mathsf{A}\boldsymbol{x} = \boldsymbol{b}. \tag{6}$$

- (**3 marks**) Solve the linear system (6) by backward substitution, using pen and paper, in the specific case where A and $\boldsymbol{b}$ are given by

```
A = [1.0 2.0 3.0; 0.0 4.0 5.0; 0.0 0.0 6.0]
b = [6.0; 9.0; 6.0]
```

- (**4 marks**) Suppose that A is upper triangular. Write a function `backward_subs(A, b)` implementing the backward substitution algorithm for solving (6) in this case.
- (**3 marks**) How many floating point operations (additions, subtractions, multiplications, divisions) does your implementation of backward substitution perform in order to solve a linear system of size n? Include the details of your calculation.
- (**Bonus, 2 marks**) Assume now that A is a banded upper triangular matrix with bandwidth 1, which means that nonzero entries are only on the main diagonal and first superdiagonal. Write a function `banded_backward_subs(A, b)` that implements an efficient version of backward substitution for this case.

Solution.

- Starting from the last equation,

$$x_3 = \frac{6}{6} = 1, \qquad x_2 = \frac{9 - 5x_3}{4} = 1, \qquad x_1 = \frac{6 - 2x_2 - 3x_3}{1} = 1.$$

The solution is $\boldsymbol{x} = (1, 1, 1)^T$.

- ```
  function backward_subs(A, b)
      n = size(A, 1)
      x = zeros(n)
      for i in n:-1:1
  ```
  ```

```
            s = b[i]
            for j in i+1:n
                s -= A[i, j] * x[j]
            end
            x[i] = s / A[i, i]
        end
        return x
    end
```

- For row i, the implementation performs $n - i$ multiplications, $n - i$ subtractions and one division. The total number of floating point operations is therefore

$$\sum_{i=1}^{n} \big(2(n-i)+1\big) = 2\sum_{k=0}^{n-1} k + n = n(n-1) + n = n^2.$$

- Only the entry $A_{i,i+1}$ is nonzero to the right of the diagonal in row $i < n$, so each step costs $\mathcal{O}(1)$ operations and the total cost is $3n - 2 = \mathcal{O}(n)$:

```
function banded_backward_subs(A, b)
    n = size(A, 1)
    x = zeros(n)
    x[n] = b[n] / A[n, n]
    for i in n-1:-1:1
        x[i] = (b[i] - A[i, i+1] * x[i+1]) / A[i, i]
    end
    return x
end
```

△

Question 5 (Iterative method for nonlinear equations, **10 marks**)**.** Consider the problem of finding a root of a nonlinear function $f\colon \mathbf{R} \to \mathbf{R}$.

- (**2 marks**) Assume that a numerical scheme of the form

$$x_{k+1} = F(x_k) \tag{7}$$

  is employed to find a root of f, where $F\colon \mathbf{R} \to \mathbf{R}$ is a given function. Assume additionally that F is globally Lipschitz continuous with Lipschitz constant $L < 1$, and that there exists a fixed point x_* of F. First, prove that the fixed point x_* is unique.

- (**3 marks**) Under the same assumptions as in the previous item, prove that the sequence of iterates $(x_k)_{k\in\mathbf{N}}$ converges to x_* for any initial guess x_0.

- (**3 marks**) We wish to compute $x_* = \sqrt[4]{2}$. Propose an iterative scheme of the form (7) to approximate this value. Write down the expression of the function F you propose. Is the convergence of your scheme superlinear? Justify your answer by examining the value of $F'(x_*)$.

- (**2 marks**) Illustrate graphically the convergence of your scheme from the previous item, for the

specific case where the initial guess is $x_0 = 1$.

Solution.

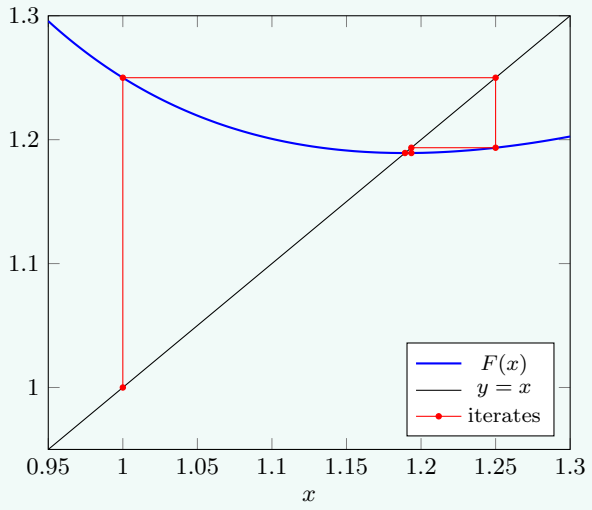
- Suppose that x_* and y_* are two fixed points of F. Then $|x_* - y_*| = |F(x_*) - F(y_*)| \leqslant L|x_* - y_*|$, so $(1 - L)|x_* - y_*| \leqslant 0$. Since $L < 1$, this implies that $x_* = y_*$.
- For all $k \in \mathbf{N}$, it holds that

$$|x_{k+1} - x_*| = |F(x_k) - F(x_*)| \leqslant L|x_k - x_*|.$$

  By induction, $|x_k - x_*| \leqslant L^k|x_0 - x_*|$, which tends to 0 as $k \to \infty$ because $L < 1$.
- A natural choice is the Newton–Raphson method applied to $f(x) = x^4 - 2$, which corresponds to

$$F(x) = x - \frac{x^4 - 2}{4x^3} = \frac{3x}{4} + \frac{1}{2x^3}.$$

  Then $F'(x) = \frac{3}{4} - \frac{3}{2x^4}$, and so $F'(x_*) = \frac{3}{4} - \frac{3}{2\times 2} = 0$. Since $F(x_*) = x_*$ and $F'(x_*) = 0$, a Taylor expansion gives $x_{k+1} - x_* = F(x_k) - F(x_*) = \frac{1}{2}F''(\xi_k)(x_k - x_*)^2$, so the convergence is superlinear (in fact quadratic) for x_0 sufficiently close to x_*.
- The iterates are obtained graphically by the "cobweb" construction: starting from (x_0, x_0) on the diagonal, one moves vertically to the graph of F, at $(x_0, F(x_0)) = (x_0, x_1)$, then horizontally back to the diagonal, at (x_1, x_1), and so on. Starting from $x_0 = 1$, we obtain $x_1 = 1.25$, $x_2 \approx 1.1935$, $x_3 \approx 1.18923$, which converge rapidly to $x_* = \sqrt[4]{2} \approx 1.189207$:



△

Implementation exercise 2 (Iterative methods for nonlinear equations, **10 marks**)**.** We wish to calculate the point $(x_*, y_*) \in \mathbf{R}^2$ on the parabolic curve $y = x^2$ that is closest to the point $(3, 1)$.

- (**2 marks**) Find a function $f\colon \mathbf{R} \to \mathbf{R}$ such that the root of f corresponds to the x-coordinate of this point.
- (**2 marks**) Let ε denote the dual unit satisfying $\varepsilon^2 = 0$. Calculate $f(x + \varepsilon)$ for $x \in \mathbf{R}$, and deduce from the result an expression for $f'(x)$.

- (**4 marks**) Implement the Newton–Raphson method to find an approximation of the root of a general differentiable function $g\colon \mathbf{R} \to \mathbf{R}$, starting from the initial guess x_0. Use an appropriate stopping criterion, and use automatic differentiation to compute the derivative g'. Your function should return only the final approximation of the root.

  Hint: Recall that `ForwardDiff.derivative(g, x)` computes the derivative of g at x.

- (**2 marks**) Use your implementation of the Newton–Raphson method (i.e., call the function with appropriate arguments) to find an approximation of the point (x_*, y_*) introduced above, and print the result.

- (**Bonus, 2 marks**) The secant method is an iterative method for finding a root of a function $f\colon \mathbf{R} \to \mathbf{R}$, where each iteration depends on the two previous iterates, according to the formula

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}\,.$$

  Implement the secant method to find an approximation of the root of f, starting from the initial guesses x_0 and x_1.

Solution.

- The squared distance between (x, x^2) and $(3, 1)$ is $d(x) = (x-3)^2 + (x^2-1)^2$, and its minimizer is a root of $\frac{1}{2}d'(x) = (x-3) + 2x(x^2-1)$. We can therefore take

$$f(x) = 2x^3 - x - 3.$$

  Since f is increasing for $|x| \geqslant \frac{1}{\sqrt{6}}$ and $f(x) < 0$ for $|x| \leqslant \frac{1}{\sqrt{6}}$, this function has a unique real root, which is the minimizer of d.

- Using $\varepsilon^2 = \varepsilon^3 = 0$,

$$f(x+\varepsilon) = 2(x^3 + 3x^2\varepsilon) - (x+\varepsilon) - 3 = (2x^3 - x - 3) + (6x^2 - 1)\varepsilon,$$

  and so $f'(x) = 6x^2 - 1$.

- ```julia
  using ForwardDiff
  function newton_raphson(f, x0; tol = 1e-12, maxiter = 10_000)
      x = x0
      for _ in 1:maxiter
          dx = f(x) / ForwardDiff.derivative(f, x)
          x -= dx
          if abs(dx) < tol
              return x
          end
      end
      error("Newton-Raphson did not converge")
  end
  ```

- ```julia
  f(x) = 2x^3 - x - 3
  x = newton_raphson(f, 1.0)
  ```

```
println((x, x^2))  # (1.289623901485061, 1.6631298072815504)
```

- ```
  function secant(f, x0, x1; tol = 1e-12, maxiter = 10_000)
      for _ in 1:maxiter
          x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
          if abs(x2 - x1) < tol
              return x2
          end
          x0, x1 = x1, x2
      end
      error("The secant method did not converge")
  end
  ```

△
  ```